Web Application Frameworks

CS457

7th January 2025

Fall 2024

Midterm

Note: There are one page and four questions in total. Electronic devices are prohibited. Please switch off your mobile phones and place them inside your bag. Any electronic device on or off near you can cancel your midterm, which may also result in other strict actions. The exchange of any items, including a pen or calculator, is strictly prohibited.

Time: 2 hours **Total Marks: 40**

[Q1] **(4+3+3)**

(a) Explain the concept of middleware in Node.js with the help of code examples. Briefly explain different kinds of middleware with code examples.

(b) Where do you think the package files are installed? Explain the purpose of module.exports. Give a clear code example to export a module in another file.

(c) What is Postman used for? Give names of any five tools other than Postman.

[Q2] **(6+4)**

(a) What are callback hells? Write a piece of code to showcase callback hells. How can you avoid callback hells? Give concrete code examples

(b) What is an event loop and how does it work? Why NodeJS is single-threaded? Why is NodeJS preferred over other backend technologies like Java and PHP?

[Q3] **(10)**

Consider you are hiring for an organization against some post, and you store information entered by potential candidate to a MongoDB collection referred here as candidates. Use ejs template engine to render a HTML form on a route / endpoint referred as "/information", containing job related fields like First Name, Last Name, Age, Qualifications, Skills, Email, Post Applied for, etc. By clicking submit button, the ExpressJS-based application at the server side receives the entered information and interacts with the MongoDB to create a document in MongoDB collection candidates. Also create a route to fetch all documents from candidates collection. Apply some style to arrange the HTML form.

[Q4] **(5+5)**

(a) How can we connect MongoDB Atlas to our web-based application. Explain the complete scenario using MVC design pattern. Write code where it is necessary.

(b) What are modules in NodeJS and what are core NodeJS modules? Give specific examples of at least 5 core modules in detail.

✅ [Q1] (Total: 10 Marks – 4 + 3 + 3)

(a) Explain the concept of middleware in Node.js with code examples. Briefly explain different kinds of middleware. *(4 marks)*

Definition:
Middleware functions in Node.js (especially with Express.js) are functions that have access to the request (req), response (res), and the next function (next). They execute in sequence and are used to modify requests, handle authentication, logging, etc.

Syntax:

javascript
CopyEdit
```
function middlewareName(req, res, next) {
   // Perform operations
   next(); // Pass control to the next middleware
}
```

Example:

javascript
CopyEdit
```
const express = require('express');
const app = express();

function logger(req, res, next) {
   console.log(`${req.method} ${req.url}`);
   next(); // move to next middleware or route
}

app.use(logger);

app.get('/', (req, res) => {
   res.send('Home Page');
});
```

Types of Middleware:

1. **Application-level:** Bound to an instance of express().
2. **Router-level:** Works the same as application-level, but bound to an instance of express.Router().
3. **Built-in middleware:** e.g., express.json() and express.static().
4. **Third-party middleware:** Like morgan, cors, helmet.
5. **Error-handling middleware:** Accepts 4 arguments (err, req, res, next).

(b) Where are package files installed? Explain `module.exports` with a code example. *(3 marks)*

Package Installation:
When you run `npm install`, packages are installed inside the **`node_modules/`** folder and dependencies are listed in `package.json`.

Purpose of `module.exports`:
Used to **export functions, objects, or variables** from a file so they can be used in another file via `require()`.

Example:

math.js

```
javascript
CopyEdit
function add(a, b) {
   return a + b;
}

module.exports = { add };
```

app.js

```
javascript
CopyEdit
const math = require('./math');
console.log(math.add(3, 4)); // Output: 7
```

(c) What is Postman used for? Name five other tools. *(3 marks)*

Postman is a GUI tool for testing APIs. You can make **GET, POST, PUT, DELETE** requests to a server and examine the response, headers, and status codes.

Uses:

- Test REST APIs
- Automate API tests
- Send JSON payloads
- Check response codes and headers

Alternatives:

1. **Insomnia**
2. **Hoppscotch (was Postwoman)**
3. **Paw (Mac)**

4. **cURL (CLI-based)**
5. **HTTPie (CLI-based)**

✅ [Q2] (Total: 10 Marks – 6 + 4)

(a) What are callback hells? Give code example and how to avoid it. *(6 marks)*

Callback Hell:
Occurs when there are **multiple nested callbacks**, making code **hard to read and maintain**.

Example:

javascript
CopyEdit
```
fs.readFile('file1.txt', (err, data1) => {
    fs.readFile('file2.txt', (err, data2) => {
        fs.readFile('file3.txt', (err, data3) => {
            console.log(data1 + data2 + data3);
        });
    });
});
```

Avoiding Callback Hell:

1. Promises:

javascript
CopyEdit
```
readFilePromise('file1.txt')
    .then(data1 => readFilePromise('file2.txt'))
    .then(data2 => readFilePromise('file3.txt'))
    .then(data3 => console.log(data3));
```

2. Async/Await:

javascript
CopyEdit
```
async function readAll() {
    const data1 = await readFilePromise('file1.txt');
    const data2 = await readFilePromise('file2.txt');
    const data3 = await readFilePromise('file3.txt');
    console.log(data1 + data2 + data3);
}
```

(b) What is event loop? Why is NodeJS single-threaded and preferred over Java/PHP? *(4 marks)*

Event Loop:
The event loop is what allows Node.js to perform **non-blocking I/O operations** on a single thread using callbacks, promises, etc.

How it works:

- Keeps checking the **callback queue**
- Executes when the call stack is empty

Why Node.js is single-threaded:

- Uses **non-blocking, asynchronous architecture**
- Avoids thread management overhead

Why preferred:

- Fast I/O operations
- Handles many clients concurrently
- Uses JavaScript on both front and back end
- Huge NPM package ecosystem
- Scalable with event-driven architecture

✅ [Q3] (10 Marks)

ExpressJS App with MongoDB and EJS to Store Candidate Info

1. MongoDB Model (`models/Candidate.js`):

```
javascript
CopyEdit
const mongoose = require('mongoose');
const candidateSchema = new mongoose.Schema({
  firstName: String,
  lastName: String,
  age: Number,
  qualifications: String,
  skills: String,
  email: String,
  postApplied: String
});
module.exports = mongoose.model('Candidate', candidateSchema);
```

2. Server Setup (server.js):

javascript
CopyEdit
```
const express = require('express');
const mongoose = require('mongoose');
const Candidate = require('./models/Candidate');
const app = express();

mongoose.connect('mongodb://localhost:27017/hrDB');

app.use(express.urlencoded({ extended: true }));
app.set('view engine', 'ejs');

// Form route
app.get('/information', (req, res) => {
  res.render('form');
});

// Submit form route
app.post('/endpoint', async (req, res) => {
  const newCandidate = new Candidate(req.body);
  await newCandidate.save();
  res.send('Candidate saved successfully');
});

// View all candidates
app.get('/candidates', async (req, res) => {
  const allCandidates = await Candidate.find();
  res.json(allCandidates);
});

app.listen(3000, () => console.log('Server running on 3000'));
```

3. View (views/form.ejs):

html
CopyEdit
```
<form method="POST" action="/endpoint" style="border:1px solid #ccc; padding:15px;">
  <input type="text" name="firstName" placeholder="First Name" required><br>
  <input type="text" name="lastName" placeholder="Last Name"><br>
  <input type="number" name="age" placeholder="Age"><br>
  <input type="text" name="qualifications" placeholder="Qualifications"><br>
  <input type="text" name="skills" placeholder="Skills"><br>
  <input type="email" name="email" placeholder="Email"><br>
  <input type="text" name="postApplied" placeholder="Post Applied"><br>
  <button type="submit">Submit</button>
</form>
```

✅ [Q4] (Total: 10 Marks – 5 + 5)

(a) Connect MongoDB Atlas using MVC Design Pattern with code *(5 marks)*

Step 1: Create cluster on MongoDB Atlas

Step 2: Connect using Mongoose

```
javascript
CopyEdit
mongoose.connect('your-atlas-uri-here', {
  useNewUrlParser: true,
  useUnifiedTopology: true
});
```

MVC Components:

- **Model:** Candidate.js (same as Q3)
- **Controller:**

```
javascript
CopyEdit
exports.addCandidate = async (req, res) => {
  const candidate = new Candidate(req.body);
  await candidate.save();
  res.redirect('/candidates');
};
```

- **View:** form.ejs
- **Route:**

```
javascript
CopyEdit
const controller = require('./controllers/candidateController');
app.post('/add', controller.addCandidate);
```

(b) NodeJS Modules – 5 Core Modules with Explanation *(5 marks)*

Module	Description	Example
fs	File system operations	fs.readFileSync()
http	Create web server	http.createServer()
path	Path operations	path.basename()
os	System info	os.cpus()
events	Event handling	emitter.on()

Examples:

javascript
CopyEdit
```
const fs = require('fs');
fs.writeFileSync('hello.txt', 'Hi there!');

const http = require('http');
http.createServer((req, res) => res.end('Server')).listen(3000);

const path = require('path');
console.log(path.extname('test.txt'));

const os = require('os');
console.log(os.platform());

const events = require('events');
const emitter = new events.EventEmitter();
emitter.on('greet', () => console.log('Hello'));
emitter.emit('greet');
```

Final 1

Web Application Framework CS-457
31st January
Fall 2024
Terminal

Time Allowed: 180 minutes **Total Points:** 50

Note: There are two pages and five questions in total. You should not carry any electronic device. Any phone smartwatch or device found near you whether on or off will lead to the cancellation of your examination.

[Q1] [CLO-1] **(4+6)**

(a) What are sharding and embedded documents in MongoDB. Give examples.

(b) Write the following MongoDB queries:

(i) Count the documents in a collection
(ii) Find documents where the age is either 25, 30, or 35.
(iii) Find documents where the name starts with "A".
(iv) Retrieve only the name and email fields.
(v) Retrieve documents with age greater than 25 **AND** less than 40.
(vi) Increment the age field by 1 for all documents.

[Q2] [CLO-3] **(3+2+5)**

(a) What is the difference between server-side rendering (SSR) and client-side rendering (CSR) in React?

(b) How are CSS modules used in React to apply style to components? Give a coding example.

(c) Consider the following relationship of components. Write code to create a React application that uses the following hierarchy of components. Pass fields such as name, age, and email through props from App Component to the Header, Logo, Main, and Sidebar components. Use Proptypes to ensure that the fields are validated in such a way where name is a string, age is a number, and email is a string.

```
App Component (Parent)
 |
 ├── Header Component (Child 1)
 |    ├── Logo Component (Grandchild 1)
 |    └── Navbar Component (Grandchild 2)
 |
 └── Main Component (Child 2)
      ├── Sidebar Component (Grandchild 3)
      └── Content Component (Grandchild 4)
```

[Q3] [CLO_1] **(5+5)**

(a) What is a template engine? Name any 5 template engines other than ejs, pug, and handlebars.

(b) What will be the output of the following code:

```
console.log("Start");
const promise1 = new Promise((resolve, reject) => { console.log("Promise 1 created");
  setTimeout(() => {  resolve("Promise 1 resolved");  }, 2000); });
const promise2 = new Promise((resolve, reject) => {  console.log("Promise 2 created");
  setTimeout(() => {  resolve("Promise 2 resolved");}, 1000);});
async function asyncFunction() {
  console.log("Async function started");
  try {   const result1 = await promise1;
   console.log(result1);
   const result2 = await promise2;
```

```
    console.log(result2);
    setTimeout(() => {     console.log("Timeout inside async function");    }, 500);
  } catch (error) {    console.log("Error:", error);  }
}
promise1
 .then((result) => {    console.log(result);
   return promise2;  })
 .then((result) => {
   console.log(result);
   setTimeout(() => {     console.log("Timeout inside promise chain");    }, 1000);
 })
 .catch((error) => {    console.log("Error:", error);  });
asyncFunction();
console.log("End");
```

[Q4] [CLO-2] **(3+3+4+3)**

(a) Create an Express router for a blog application with routes for creating, reading, updating, and deleting blog posts.

(b) Write an Express route that uses async/await to fetch data from a MongoDB collection and return it as a JSON response.

(c) Write an Express app that connects to a MongoDB database using Mongoose. Include routes for creating, reading, updating, and deleting (CRUD) documents in a users collection.

(d) Explain how middleware chaining works in Express.js. Write a custom middleware that logs the request method and URL, and then passes control to the next middleware.

[Q5] [CLO-3] **(7)**

Consider the following bbc website and create a replica of this using React. Use at least 4 to 6 components.

[Q1] [CLO-1]

(a) What are sharding and embedded documents in MongoDB? Give examples.

- **Sharding**: This is a method used in MongoDB to horizontally partition data across multiple servers or shards to handle large datasets and high throughput. It improves scalability and performance by distributing the load. For instance, in a "users" collection with millions of records, sharding by "userId" ensures each shard contains a subset of users, reducing the load on any single server.
    - Example: { _id: "user1", userId: "001", name: "John" } on one shard, and { _id: "user2", userId: "002", name: "Jane" } on another.
- **Embedded Documents**: These are documents nested within another document, allowing for a structured representation of related data. This is useful for one-to-many relationships without requiring separate collections.
    - Example: { name: "John", address: { city: "New York", zip: 10001, country: "USA" } }.

(b) Write the following MongoDB queries:

(i) Count the documents in a collection:
db.collection.countDocuments();

This command returns the total number of documents in the specified collection, e.g., db.users.countDocuments() would count all user records.

(ii) Find documents where the age is either 25, 30, or 35:
db.collection.find({ age: { $in: [25, 30, 35] } });

The $in operator matches any document where the age field equals one of the specified values.

(iii) Find documents where the name starts with "A":
db.collection.find({ name: /^A/ });
The ^A regex pattern matches strings beginning with "A", e.g., "Alice" or "Alex".

(iv) Retrieve only the name and email fields:
db.collection.find({}, { name: 1, email: 1, _id: 0 });

The projection { name: 1, email: 1, _id: 0 } includes name and email while excluding the default _id field.
(v) Retrieve documents with age greater than 25 AND less than 40:
db.collection.find({ age: { $gt: 25, $lt: 40 } });

The $gt (greater than) and $lt (less than) operators filter documents within the range 26 to 39.

(vi) Increment the age field by 1 for all documents:
db.collection.updateMany({}, { $inc: { age: 1 } });

The $inc operator increases the age field by 1 for every document in the collection.

[Q2] [CLO-3]

(a) What is the difference between server-side rendering (SSR) and client-side rendering (CSR) in React?

- **Server-Side Rendering (SSR)**: The server generates the initial HTML for a page and sends it to the client. This improves SEO (search engine optimization) and initial load performance, especially for content-heavy sites. Frameworks like Next.js implement SSR, where the server pre-renders pages based on routes.
- **Client-Side Rendering (CSR)**: The browser downloads a minimal HTML page and uses JavaScript (e.g., React) to render the full UI dynamically. This offers a smoother user experience with real-time updates but may impact initial load time and SEO. A typical Create React App project uses CSR.

(b) How are CSS modules used in React to apply style to components? Give a coding example.

- **CSS Modules**: These allow locally scoped CSS by generating unique class names, preventing style conflicts across components. You import the CSS file as an object and use its properties as class names.
    - Example:

```
/* styles.module.css */

.container { color: blue; font-size: 16px; }

.title { font-weight: bold; }

// Component.jsx

import styles from './styles.module.css';

function MyComponent() {

  return (

    <div className={styles.container}>

      <h1 className={styles.title}>Hello World</h1>

    </div>

  );
```

```
}

export default MyComponent;
```

- The styles are scoped to MyComponent, avoiding global namespace pollution.

(c) Consider the following hierarchy of components. Write code to create a React application that uses the following hierarchy of components. Pass fields such as name, age, and email through props from App component to the Header, Logo, Main, and Sidebar components. Use PropTypes to ensure that the fields are validated in such a way where name is a string, age is a number, and email is a string.

```
import PropTypes from 'prop-types';



const Header = ({ name, age, email }) => (

  <header>

    <h1>User Info: {name} - Age: {age} - Email: {email}</h1>

  </header>

);

Header.propTypes = {

  name: PropTypes.string.isRequired,

  age: PropTypes.number.isRequired,

  email: PropTypes.string.isRequired,

};



const Logo = ({ name }) => <div className="logo">{name} Logo</div>;

Logo.propTypes = { name: PropTypes.string.isRequired };



const Navbar = () => <nav><ul><li>Home</li><li>About</li></ul></nav>;
```

```
const Main = ({ age }) => <main><p>Current Age: {age}</p></main>;

Main.propTypes = { age: PropTypes.number.isRequired };


const Sidebar = ({ email }) => <aside><p>Contact: {email}</p></aside>;

Sidebar.propTypes = { email: PropTypes.string.isRequired };


const App = () => {

  const userData = { name: "John Doe", age: 30, email: "john@example.com" };

  return (

    <div className="app">

      <Header {...userData} />

      <Logo name={userData.name} />

      <Navbar />

      <div className="content">

        <Main age={userData.age} />

        <Sidebar email={userData.email} />

      </div>

    </div>

  );

};


export default App;
```

- **Explanation**: The App component passes props to child components (Header, Logo, Main, Sidebar) using the spread operator ({...userData}). PropTypes ensures type validation, throwing warnings in development if the types mismatch (e.g., passing a string for age).

[Q3] [CLO-1]

(a) What is a template engine? Name any 5 template engines other than ejs, pug, and handlebars.

- **Template Engine**: A tool that processes templates with placeholders and data to generate dynamic HTML. It separates presentation from logic, making web development more maintainable.
- **Examples**:
  - **Jinja2**: Used with Python (Flask), supports loops and conditionals.
  - **Mustache**: Logic-less, works with multiple languages (e.g., Node.js, PHP).
  - **Haml**: Concise, indentation-based syntax for Ruby on Rails.
  - **Liquid**: Popular with Shopify, supports filters and tags.
  - **Thymeleaf**: Java-based, integrates well with Spring.

(b) What will be the output of the following code:

```
const promise1 = new Promise((resolve, reject) => {

  console.log("Promise 1 created");

  setTimeout(() => { resolve("Promise 1 resolved"); }, 2000);

});

const promise2 = new Promise((resolve, reject) => {

  console.log("Promise 2 created");

  setTimeout(() => { resolve("Promise 2 resolved"); }, 1000);

});

async function asyncFunction() {

  console.log("Async function started");

  try {

    const result1 = await promise1;

    console.log(result1);

    const result2 = await promise2;

    console.log(result2);

  } catch (error) {
```

```
    console.log("Error:", error);

  }

  console.log("End");

}

asyncFunction();
```

- **Output Explanation**:
  - **Initial Logs**: "Promise 1 created", "Promise 2 created", and "Async function started" print immediately as they are synchronous.
  - **Async Await**: await promise1 pauses execution for 2 seconds, then logs "Promise 1 resolved". await promise2 waits for the next 1 second (though promise2 resolved earlier), logging "Promise 2 resolved".
  - **Final Log**: "End" prints after all awaits complete.
  - **Sequence**:

    text

    CollapseWrap

    Copy

    ```
    Promise 1 created

    Promise 2 created

    Async function started

    Promise 1 resolved  (after 2s)

    Promise 2 resolved  (after 1s, but waits for promise1)

    End
    ```

  - Note: promise2 resolves after 1s, but await processes promises sequentially.

[Q4] [CLO-2]

(a) Create an Express router for a blog application with routes for creating, reading, updating, and deleting blog posts.

```
const express = require('express');

const router = express.Router();


router.get('/posts', (req, res) => {

  res.send('List of all blog posts');

}); // Read all posts


router.post('/posts', (req, res) => {

  res.send('New blog post created');

}); // Create a post


router.put('/posts/:id', (req, res) => {

  res.send(`Blog post ${req.params.id} updated`);

}); // Update a post


router.delete('/posts/:id', (req, res) => {

  res.send(`Blog post ${req.params.id} deleted`);

}); // Delete a post


module.exports = router;
```

- **Explanation**: Each route handles a CRUD operation, with :id as a parameter for specific post manipulation.

(b) Write an Express route that uses async/await to fetch data from a MongoDB collection and return it as a JSON response.

```
const express = require('express');
```

```
const router = express.Router();

const mongoose = require('mongoose');


const DataSchema = new mongoose.Schema({ name: String, value: Number });

const Data = mongoose.model('Data', DataSchema);


router.get('/data', async (req, res) => {

  try {

    const data = await Data.find(); // Fetch all documents

    res.json(data); // Return as JSON

  } catch (error) {

    res.status(500).json({ error: 'Failed to fetch data', details:
error.message });

  }

});


module.exports = router;
```

- **Explanation**: The route uses async/await to handle asynchronous MongoDB queries, returning data or an error response.

(c) Write an Express app that connects to a MongoDB database using Mongoose. Include routes for creating, reading, updating, and deleting (CRUD) documents in a users collection.

```
const express = require('express');

const mongoose = require('mongoose');

const app = express();
```

```
app.use(express.json()); // Parse JSON bodies


mongoose.connect('mongodb://localhost/usersdb', {
  useNewUrlParser: true,
  useUnifiedTopology: true
}).then(() => console.log('Connected to MongoDB'))
  .catch(err => console.error('Connection error:', err));


const UserSchema = new mongoose.Schema({
  name: { type: String, required: true },
  age: { type: Number, required: true }
});
const User = mongoose.model('User', UserSchema);


// CRUD Routes
app.get('/users', async (req, res) => {
  const users = await User.find();
  res.json(users);
});


app.post('/users', async (req, res) => {
  const user = new User(req.body);
  await user.save();
  res.status(201).json(user);
});
```

```
app.put('/users/:id', async (req, res) => {
  const user = await User.findByIdAndUpdate(req.params.id, req.body, { new:
true, runValidators: true });
  if (!user) return res.status(404).json({ error: 'User not found' });
  res.json(user);
});

app.delete('/users/:id', async (req, res) => {
  const user = await User.findByIdAndDelete(req.params.id);
  if (!user) return res.status(404).json({ error: 'User not found' });
  res.json({ message: 'User deleted' });
});

app.listen(3000, () => console.log('Server running on port 3000'));
```

- **Explanation**: The app connects to MongoDB, defines a User model, and implements full CRUD operations with error handling.

(d) Explain how middleware chaining works in Express.js. Write a custom middleware that logs the request method and URL, and then passes control to the next middleware.

- **Middleware Chaining**: In Express.js, middleware functions are executed in the order they are defined using app.use() or router.use(). Each middleware can process the request, modify it, or pass control to the next function via next(). This allows for modular request handling (e.g., logging, authentication).
- **Example:**

```
const express = require('express');

const app = express();
```

```
const logMiddleware = (req, res, next) => {

  console.log(`[${new Date().toISOString()}] ${req.method}
${req.url}`);

  next(); // Passes control to the next middleware or route handler

};



const authMiddleware = (req, res, next) => {

  console.log('Auth check');

  next();

};

app.use(logMiddleware);

app.use(authMiddleware);

app.get('/', (req, res) => {

  res.send('Home Page');

});

app.listen(3000, () => console.log('Server started'));
```

- **Explanation**: logMiddleware logs the request details, and authMiddleware checks authentication, both passing control with next().

[Q5] [CLO-3]

Consider the following BBC website and create a replica of this using React. Use at least 4 to 6 components.

```
import React from 'react';

import './App.css';


const Header = () => (

  <header className="header">

    <div className="logo">BBC</div>

    <div className="auth">

      <a href="#">Register</a> | <a href="#">Sign in</a>

    </div>

  </header>

);


const Navbar = () => (

  <nav className="navbar">

    <a href="#">Home</a> | <a href="#">News</a> | <a href="#">Sport</a> | <a
href="#">Business</a> |

    <a href="#">Innovation</a> | <a href="#">Culture</a> | <a
href="#">Arts</a> | <a href="#">Travel</a> |

    <a href="#">Video</a> | <a href="#">Live</a>

  </nav>

);


const Main = () => (

  <main className="main">

    <h2>Business</h2>
```

```
    <article>
      <h3>US economic growth slows despite consumers spending more</h3>
      <p>The world's largest economy missed forecasts after expanding at 2.8% between October and December.</p>
    </article>
    <article>
      <h3>Meta to pay $25m to settle Trump lawsuit over ban</h3>
      <p>The former president sued the firm in 2021 over the suspension of his accounts after the 6 January Capitol riots.</p>
    </article>
  </main>
);

const Sidebar = () => <aside className="sidebar">Sidebar Content</aside>;

const Footer = () => <footer className="footer">© BBC 2025</footer>;

const App = () => (
  <div className="app">
    <Header />
    <Navbar />
    <div className="content">
      <Main />
      <Sidebar />
    </div>
```

```
    <Footer />
  </div>
);


export default App;
/* App.css */
.app { font-family: Arial, sans-serif; max-width: 1200px; margin: 0 auto; }
.header { display: flex; justify-content: space-between; padding: 10px;
border-bottom: 1px solid #ccc; }
.logo { font-weight: bold; font-size: 24px; }
.auth a { margin-left: 10px; text-decoration: none; }
.navbar { padding: 10px; background: #f8f8f8; }
.navbar a { margin: 0 5px; text-decoration: none; }
.content { display: flex; }
.main { flex: 3; padding: 20px; }
.main h2 { margin-top: 0; }
.main article { margin-bottom: 20px; }
.sidebar { flex: 1; padding: 20px; background: #f0f0f0; }
.footer { text-align: center; padding: 10px; border-top: 1px solid #ccc; }
```

- **Explanation**: The app uses six components (Header, Navbar, Main, Sidebar, Footer, App) to replicate the BBC layout. CSS styles ensure a responsive two-column layout with a header, navigation, main content, sidebar, and footer.

Final 2

[Q1] (2+3+5)

(a) What is JSX? Why can't browsers read JSX?

(b) What are the advantages and limitations of ReactJS?

(c) What does DOM stand for? What do you understand by Virtual DOM? Explain its working. Also differentiate between real DOM and Virtual DOM?

[Q2] (4+3+3)

(a) What are React hooks? What are the benefits of using React hooks? What is the useState hook and how it is used?

(b) What do you mean by the phenomena of props drilling? Give some concrete examples.

(c) Why do we need a Router in React? What is nested routing in React? Give concrete examples.

[Q3] (3+4+3)

(a) What is Postman used for? Give names of any alternative five tools other than Postman and ThunderClient.

(b) What are design patterns? What other design patterns (at least five others than MVC) can be used to build a web-based application? Briefly explain all of them.

(c) What is the major difference between Rest and GraphQL API design architecture? What REST limitations does GraphQL attempt to overcome?

[Q4] (4+3+3)

(a) What is callback hell? Explain with the help of a concrete example. What are the alternative options to callback hell? Give concrete examples.

(b) What is an event loop and how does it work?

(c) Explain the concept of middleware in NodeJS? Explain with the help of examples.

[Q5] (5+5)

(a) How can we connect MongoDB Atlas to our web-based application. Explain the complete scenario using MVC design pattern. Write code where it is necessary.

(b) What are modules in NodeJS and what are core NodeJS modules? Give specific examples of at least 5 core modules in detail.

[Q1] [CLO-?] (2+3+5)

(a) What is JSX? Why can browsers read JSX?

- **JSX**: JSX (JavaScript XML) is a syntax extension for JavaScript, primarily used with React, that allows developers to write HTML-like code within JavaScript. It enhances readability and enables the creation of reusable UI components by combining markup and logic. For example, Hello {name} is a JSX expression that gets compiled into JavaScript.
- **Why Browsers Read JSX**: Browsers cannot natively interpret JSX because it is not valid JavaScript. Instead, tools like Babel transpile JSX into standard JavaScript (e.g., React.createElement('div', null, 'Hello ', name)) during the build process, allowing browsers to execute the resulting code.

(b) What are the advantages and limitations of ReactJS?

- **Advantages**:
  - **Component Reusability**: Components can be reused across the application, reducing redundancy (e.g., a Button component used in multiple places).
  - **Virtual DOM**: Improves performance by minimizing direct DOM manipulations through efficient updates.
  - **Strong Ecosystem**: Rich libraries (e.g., Redux, React Router) and a large community support development.
- **Limitations**:
  - **Steep Learning Curve**: Concepts like hooks and state management can be challenging for beginners.
  - **SEO Challenges**: Client-side rendering (CSR) may hinder search engine indexing unless server-side rendering (SSR) is implemented.
  - **Large Bundle Size**: Initial load can be slow for small projects due to the included JavaScript.

(c) What does DOM stand for? What do you understand by Virtual DOM? Explain its working. Also differentiate between real DOM and Virtual DOM?

- **DOM**: Stands for Document Object Model, a structured representation of HTML documents as a tree of nodes (elements, attributes, text) managed by the browser. It allows JavaScript to dynamically interact with and modify the page.
- **Virtual DOM**: A lightweight, in-memory copy of the real DOM maintained by React. It serves as an intermediary layer to optimize rendering by tracking changes and updating only the necessary parts of the UI.
- **Working**:
  1. React creates a Virtual DOM tree representing the UI.
  2. When state or props change, a new Virtual DOM tree is generated.
  3. React compares the new and old trees (diffing) to identify differences.

4. It applies only the necessary updates to the real DOM via reconciliation, minimizing performance overhead.

- **Difference**:
  - **Real DOM**: Managed directly by the browser, updates trigger reflow and repaint, which can be slow for large changes.
  - **Virtual DOM**: An abstract layer in memory, updates are fast as they avoid direct DOM manipulation until the diffing process is complete.

[Q2] [CLO-?] (4+3+3)

(a) What are React hooks? What are the benefits of using React hooks? What is the useState hook and how is it used?

- **React Hooks**: Introduced in React 16.8, hooks are functions that enable state and lifecycle features in functional components, eliminating the need for class components. Examples include useState, useEffect, and useContext.
- **Benefits**:
  - **Logic Reuse**: Allows stateful logic to be extracted into custom hooks for reuse across components.
  - **Simplified Code**: Reduces boilerplate compared to class-based components.
  - **Better Readability**: Organizes related code (e.g., state and effects) together rather than scattering it across lifecycle methods.
- **useState**: A hook that declares and manages state in functional components, returning a state value and a function to update it.
  - **Usage**:

```
import React, { useState } from 'react';

function Counter() {

  const [count, setCount] = useState(0); // Initializes state to
0

  return (

    <button onClick={() => setCount(count + 1)}>

      Count: {count}

    </button>

  );
```

```
}

export default Counter;
```

- **Explanation**: useState(0) returns an array with the current state (count) and the updater function (setCount). Clicking the button increments the count.

(b) What do you mean by the phenomena of props drilling? Give some concrete examples.

- **Props Drilling**: The process of passing props through multiple layers of components, even if intermediate components don't need them, to reach a deeply nested component. This can lead to cluttered code and maintenance issues.
- **Examples**:
  - **Simple Example**:

```
function App() { return <Header userData="John" />; }

function Header({ userData }) { return <Profile
userData={userData} />; }

function Profile({ userData }) { return <div>User:
{userData}</div>; }
```

    - Here, userData is passed from App to Header to Profile, even though Header doesn't use it.
  - **Complex Example**: Passing user authentication data through multiple layout components to a specific form.
  - **Mitigation**: Use Context API or custom hooks to avoid drilling.

(c) Why do we need a Router in React? What is nested routing in React? Give concrete examples.

- **Need for Router**: In single-page applications (SPAs), a router (e.g., React Router) enables navigation between different views or components without full page reloads, providing a seamless user experience and managing URL changes.
- **Nested Routing**: Allows defining routes within other routes, enabling sub-sections or child components to have their own navigation. This is useful for complex UIs like dashboards.
- **Examples**:

```
import { BrowserRouter, Route, Routes, Link } from 'react-router-dom';

function Home() { return <h1>Home</h1>; }

function Dashboard() { return <h1>Dashboard</h1>; }
```

```
function Profile() { return <h1>Profile</h1>; }

function App() {

  return (

    <BrowserRouter>

      <nav>

        <Link to="/">Home</Link> | <Link
to="/dashboard">Dashboard</Link>

      </nav>

      <Routes>

        <Route path="/" element={<Home />} />

        <Route path="/dashboard" element={<Dashboard />}>

          <Route path="profile" element={<Profile />} />

        </Route>

      </Routes>

    </BrowserRouter>

  );

}

export default App;
```

- **Explanation**: /dashboard/profile nests the Profile route under Dashboard, accessible via the URL.

[Q3] [CLO-?] (3+4+3)

(a) What is Postman used for? Give names of any alternative five tools other than Postman and ThunderClient.

- **Postman**: A popular API development tool used for designing, testing, documenting, and monitoring APIs. It allows sending HTTP requests (GET, POST, etc.), viewing responses, and automating tests with collections.
- **Alternatives**:
  - **Insomnia**: A lightweight alternative with a user-friendly interface for API testing.
  - **Paw**: Offers advanced features like code generation and environment management.
  - **SoapUI**: Focused on testing SOAP and REST APIs with robust scripting capabilities.
  - **cURL**: A command-line tool for transferring data with URLs, widely used for quick API calls.
  - **HTTPie**: A user-friendly command-line HTTP client with a simpler syntax than cURL.

(b) What are design patterns? What other design patterns (at least five others than MVC) can be used to build a web-based application? Briefly explain all of them.

- **Design Patterns**: Proven templates for solving common software design problems, providing reusable and efficient solutions.
- **Other Patterns**:
  - **Singleton**: Ensures a class has only one instance, useful for managing shared resources like database connections.
    - Example: A single configuration manager.
  - **Factory**: Creates objects without specifying the exact class, promoting flexibility in object creation.
    - Example: Creating different types of API clients.
  - **Observer**: Defines a one-to-many dependency where objects are notified of state changes.
    - Example: Real-time updates in a chat application.
  - **Decorator**: Adds responsibilities to objects dynamically without modifying their code.
    - Example: Adding logging to a service.
  - **Strategy**: Defines a family of algorithms, making them interchangeable within a context.
    - Example: Different sorting strategies in a data processor.

(c) What is the major difference between REST and GraphQL API design architecture? What REST limitations does GraphQL attempt to overcome?

- **Difference**:
  - **REST**: Relies on multiple fixed endpoints (e.g., /users, /posts) and returns predefined data structures, often leading to over-fetching or under-fetching.
  - **GraphQL**: Uses a single endpoint with flexible queries, allowing clients to request exactly the data they need in a single call.
- **Limitations Overcome**:
  - **Over-fetching/Under-fetching**: REST may return more or less data than needed; GraphQL resolves this with custom queries.
  - **Multiple Requests**: REST often requires multiple calls for related data; GraphQL fetches it in one request.

- **Versioning**: REST APIs may need versioning (e.g., /v1/, /v2/); GraphQL avoids this by evolving the schema.

[Q4] [CLO-?] (4+3+3)

(a) What is callback hell? Explain with the help of a concrete example. What are the alternative options to callback hell?

- **Callback Hell**: Occurs when multiple nested callbacks are used to handle asynchronous operations, leading to unreadable and hard-to-maintain code, often resembling a "pyramid of doom."
- **Example**:

```
getUserData(function(user) {

  getOrders(user.id, function(orders) {

    processOrders(orders, function(total) {

      console.log(`Total: ${total}`);

    });

  });

});
```

  - This nested structure is difficult to debug and extend.
- **Alternatives**:
  - **Promises**: Chain .then() calls for sequential async operations.
  - **Async/Await**: Syntactic sugar over promises for cleaner async code.
  - **Observables**: Used with libraries like RxJS for handling multiple async events.

(b) What is an event loop and how does it work?

- **Event Loop**: A core concept in Node.js that manages asynchronous operations, ensuring non-blocking I/O. It continuously checks the call stack and event queue to execute callbacks when the stack is empty.
- **Working**:
  - The call stack executes synchronous code.
  - Asynchronous tasks (e.g., file I/O, network requests) are delegated to the libuv library and placed in the event queue.

- Once the call stack is clear, the event loop picks tasks from the queue and pushes their callbacks to the stack for execution.
- This cycle repeats, enabling efficient handling of concurrent operations.

(c) Explain the concept of middleware in Node.js? Explain with the help of examples.

- **Middleware**: Functions in Node.js (e.g., in Express) that process HTTP requests between the server and the final route handler. Middleware can modify requests, add headers, log data, or terminate the request-response cycle.
- **Example**:

  javascript

  CollapseWrapRun

  Copy

```javascript
const express = require('express');

const app = express();



// Logging middleware

app.use((req, res, next) => {

  console.log(`Request received at ${new Date()} for ${req.url}`);

  next(); // Passes control to the next middleware or route

});



// Route handler

app.get('/', (req, res) => {

  res.send('Hello, World!');

});
```

```
app.listen(3000, () => console.log('Server running on port 3000'));
```

- **Explanation**: The middleware logs each request's URL and timestamp before passing control to the route handler, demonstrating request processing.

[Q5] [CLO-?] (5+5)

(a) How can we connect MongoDB Atlas to our web-based application. Explain the complete scenario using MVC design pattern. Write code where it is necessary.

- **Connection Process**:
  1. **Setup**: Sign up on MongoDB Atlas, create a cluster, and obtain the connection string (e.g., mongodb+srv://<user>:<password>@cluster0.mongodb.net/<db>?retryWrites=true&w=majority).
  2. **Install Dependencies**: Use npm install mongoose to integrate MongoDB with Node.js.
  3. **MVC Implementation**:
     - **Model**: Defines the data structure (schema).
     - **View**: Renders the UI based on data.
     - **Controller**: Handles business logic and interacts with the model.
- **Code**:

```
// Model (models/user.js)

const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({

  name: { type: String, required: true },

  age: { type: Number, required: true }

});

module.exports = mongoose.model('User', userSchema);



// Controller (controllers/userController.js)
```

```
const User = require('../models/user');

exports.getUsers = async (req, res) => {

  try {

    const users = await User.find();

    res.render('users', { users }); // Render view with user data

  } catch (error) {

    res.status(500).send('Error fetching users');

  }

};

// App (app.js)

const express = require('express');

const mongoose = require('mongoose');

const app = express();

app.set('view engine', 'ejs'); // Set EJS as view engine



mongoose.connect('mongodb+srv://<user>:<pass>@cluster0.mongodb.net/test
?retryWrites=true&w=majority', {

  useNewUrlParser: true,

  useUnifiedTopology: true

}).then(() => console.log('Connected to MongoDB Atlas'))

  .catch(err => console.error('Connection error:', err));
```

```
app.use(express.json());

app.get('/users', require('./controllers/userController').getUsers);

app.listen(3000, () => console.log('Server running on port 3000'));
```

- **View (views/users.ejs)**: <ul><%= users.forEach(user => <li>${user.name}</li>) %></ul> (simplified).

(b) What are modules in Node.js and what are core Node.js modules? Give specific examples of at least 5 core modules in detail.

- **Modules**: In Node.js, modules are reusable units of code that can be imported using require() or import. They encapsulate functionality, promoting modularity and maintainability.
- **Core Modules**: Built-in modules provided by Node.js, requiring no installation, and forming the foundation of Node.js applications.
- **Examples**:
  - **fs (File System)**: Handles file operations like reading and writing.
    - Example: const fs = require('fs'); console.log(fs.readFileSync('file.txt', 'utf8'));
  - **http**: Creates HTTP servers and handles client requests.
    - Example: const http = require('http'); http.createServer((req, res) => res.end('Hi')).listen(3000);
  - **path**: Manages file and directory paths across platforms.
    - Example: const path = require('path'); console.log(path.join(__dirname, 'data', 'file.txt'));
  - **os**: Provides operating system-related utilities.
    - Example: const os = require('os'); console.log(os.platform()); // e.g., 'linux'
  - **events**: Supports event-driven programming with an EventEmitter.
    - Example:

```
const EventEmitter = require('events');

const ee = new EventEmitter();

ee.on('greet', () => console.log('Hello'));

ee.emit('greet');
```